Operator Overloading

林致宇

Outline

- Operator is a function
- Chaining
- Overloading typecasts
- Functor (Function Object)

Outline

- Operator is a function
- Chaining
- Overloading typecasts
- Functor (Function Object)

Operator is a function

- 3 + 5 可視為 operator+(3, 5)

A simple example

A "const function", denoted with the keyword const after a function declaration, makes it a compiler error for this class function to change a member variable of the class. However, reading of a class variables is okay inside of the function, but writing inside of this function will generate a compiler error.

```
 4  class Rect {
 5  public:
 6      double length;
 7      double width;
 8      bool operator<(const Rect& rect) const {
 9          return (length*width) < (rect.length*rect.width);
10      }
11  };
12
13  int main(int argc, const char * argv[]) {
14      Rect rect1; rect1.length = 3.0; rect1.width = 3.0;
15      Rect rect2; rect2.length = 2.0; rect2.width = 5.0;
16      if (rect1 < rect2) {
17          cout << "Rect1 < Rect2" << endl;
18      }
19  }
```

不能动到 Member Variables 的值

Non Member Function

有時會需要使用到 friend

```
4  class Rect {
5  public:
6      double length;
7      double width;
8  };
9
10 bool operator < (const Rect& r1, const Rect& r2) {
11     return (r1.length*r1.width) < (r2.length*r2.width);
12 }
13
14 int main(int argc, const char * argv[]) {
15     Rect rect1; rect1.length = 3.0; rect1.width = 3.0;
16     Rect rect2; rect2.length = 2.0; rect2.width = 5.0;
17     if (rect1 < rect2) {
18         cout << "Rect1 < Rect2" << endl;
19     }
20 }
```

All Relational Operators

```
1  #include<iostream>
2  #include<utility>
3  using namespace std;
4  using namespace rel_ops;
5
6  class Rect {
7  public:
8      double length;
9      double width;
10     bool operator<(const Rect& rect) const {
11         return (length*width) < (rect.length*rect.width);
12     }
13     bool operator==(const Rect& rect) const {
14         return (length*width) == (rect.length*rect.width);
15     }
16 };
17
18 int main(int argc, const char * argv[]) {
19     Rect rect1; rect1.length = 3.0; rect1.width = 3.0;
20     Rect rect2; rect2.length = 2.0; rect2.width = 4.0;
21     if (rect1 > rect2) {
22         cout << "Rect1 > Rect2" << endl;
23     }
24 }
```

Practice

- Calendar
  - Year
  - Month
  - Day

- Example
  - 2023/1/1 > 2022/12/30

Operators You Can/Cannot Overload

- https://www.ibm.com/docs/en/zos/2.4.0?topic=only-overloading-operators-c

- 不可多載定義的運算子： :: , . , sizeof , .* , ?:

- 不能重新定義內建資料型別的運算方式

(More Example) Unary Operator

```
 4  class N {
 5  public:
 6      int number;
 7
 8      bool operator!() {
 9          if (number%2 == 0) { return 0; }
10          else { return 1; }
11      }
12  };
13
14  int main(int argc, const char * argv[]) {
15      N n;
16      n.number = 7;
17      cout << !n << endl;
18      n.number = 6;
19      cout << !n << endl;
20  }
```

(More Example) [] Operator

```
#include<iostream>
#include<cassert>
using namespace std;

class I {
public:
    int number;
    int operator[](int i);
};

int I::operator[](int i) {
    assert(i >= 0);
    int x = 10;
    for (int j = 0; j < i; j++) {
        x *= 10;
    }
    int result = number%x;
    result = result/(x/10);
    return result;
}
```

```
int main() {
    I i;
    i.number = 57013;
    for (int j = 0; j < 5; j++) {
        cout << i[j] << "\t";
    }
    cout << endl;
    cout << i[5] << endl;
    cout << i[-1] << endl;
}
```

(More Example) Assignment Operator

```
 4  class I {
 5  public:
 6      int number;
 7      I operator=(const int i);
 8  };
 9
10  I I::operator=(const int i) {
11      this->number = i;
12      return *this;
13  }
14
15  int main(int argc, const char * argv[]) {
16      I i;
17      i = 7;
18      cout << i.number << endl;
19  }
```

Outline

- Operator is a function
- Chaining
- Overloading typecasts
- Functor (Function Object)

Chaining (<<)

```
 4  class Person {
 5  public:
 6      int money;
 7      Person(int m) {money = m;}
 8      Person operator+(const Person& p) const {
 9          return Person(money+p.money);
10      }
11  };
12
13  int main(int argc, const char * argv[]) {
14      Person p1(5); Person p2(3); Person p3(2);
15      Person p = p1 + p2 + p3;
16      cout << p.money << endl;
17  }
```

Outline

- Operator is a function
- Chaining
- Overloading typecasts
- Functor (Function Object)

Overloading typecasts

```
 4  class I {
 5  public:
 6      int number;
 7      operator int() { return number; }
 8  };
 9
10  int main() {
11      I i;
12      i.number = 7;
13      int j = (int)i + 3;
14      cout << j << endl;
15  }
```

強制転形的operator

Overloading typecasts

```
4  class Rect {
5  public:
6      double length;
7      double width;
8      operator double() const {
9          return (length*width);
10     }
11 };
12
13 int main(int argc, const char * argv[]) {
14     double d1 = 3.0;
15     Rect rect1; rect1.length = 3.0; rect1.width = 3.0;
16     double d2 = d1 + static_cast<double>(rect1);
17     cout << d2 << endl;
18 }
```

!!

強制転形(Morden C++寫法)→直接算面積!

d1 + (double)rect1;

Outline

- Operator is a function
- Chaining
- Overloading typecasts
- Functor (Function Object)

Functor (Function Object)

- https://www.geeksforgeeks.org/functors-in-cpp/
  - A functor (or function object) is a C++ class that acts like a function.

- https://openhome.cc/Gossip/CppGossip/Functor.html

Functor (Function Object)

```
4  class CircleArea {
5  public:
6      double operator()(double r){
7          return (3.14*r*r);
8      }
9  };
10
11 void getCircleArea(double r, CircleArea ca) {
12     double area = ca(r);
13     cout << area << endl;
14 }
15
16 int main(int argc, const char * argv[]) {
17     CircleArea ca;
18     getCircleArea(2.0, ca);
19 }
```

// Higher Order Function